

A Programmable 5G DU-RU SmartNIC based on MPSoC FPGA

Abdelghani Bourenane¹, Emilio Paolini^{1,2,3}, Nicola Andriolli⁴, and Luca Valcarengi¹

¹*Scuola Superiore Sant'Anna, Pisa, Italy*

²*Institute of Electronics, Computer and Telecommunication Engineering (CNR-IEIT), Pisa, Italy*

³*Sma-RTy Italia Srl, Carugate, Italy*

⁴*Department of Information Engineering (DII), University of Pisa, Pisa, Italy*

abdelghani.bourenane@santannapisa.it

Abstract—The adoption of disaggregated, virtualized, and open gNodeB in the next generation Radio Access Network offers benefits such as cost reduction and improved network performance. However, meeting specific 5G and beyond requirements, e.g., Ultra Reliable Low Latency Communications, requires offloading selected gNodeB functions onto accelerated hardware.

This study proposes to implement a 5G Distributed Unit (DU)-Radio Unit (RU) in a System-on-a-Programmable-Chip (SoPC) where the FFT of Orthogonal Frequency Division Multiplexing in uplink transmission is offloaded onto an FPGA. The proposed solution is programmable and pluggable, allowing flexibility in function implementation and integration into various devices.

The experimental evaluation shows that the proposed SmartNIC-based DU-RU achieves 15× speedup in processing time when compared to a server's CPU with FPGA-accelerated Low-PHY processing. In addition, it shows that employing high-performance off-chip memory leads to about 14% reduction in processing time compared to the use of an FPGA-internal block memory.

Index Terms—SoPC, FPGA, Memory, RU, DU, Low-PHY, FFT

I. INTRODUCTION

5G and beyond technology is emerging as a breakthrough technology, potentially revolutionizing many application fields, ranging from health to industry. The main features of this novel generation of mobile networks comprise (i) massive capacity, i.e., very high bandwidth, with order of magnitudes improvement with respect to older generations; (ii) higher frequency re-use; (iii) end-to-end network security, and (iv) new network paradigms [1]. With the advent of NextG networks, e.g., 6G, these features will be further enhanced [2].

However, current Radio Access Network (RAN) components operate as monolithic entities, serving as all-in solutions that encompass every layer of the cellular protocol stack [3]. The lack of flexibility results in limited reconfigurability of equipment, missing coordination among network nodes, and vendor lock-in, making optimized radio resource management and efficient spectrum utilization extremely challenging [4].

This research is partly supported by the project CLEVER (project number 101097560). The project is supported by the Key Digital Technologies Joint Undertaking and its members (including top-up funding by the Italian Ministry of Research and University (MUR)). This work is also partially supported by the European Union under the Italian National Recovery and Resilience Plan (NRRP) of NextGenerationEU, a partnership on "Telecommunications of the Future" (PE00000001 - program "RESTART").

To overcome these limitations, both academia and industries have promoted the concept of Open RAN, focusing on the enhancement of RAN performance exploiting virtualized network elements and open interfaces, incorporating intelligence in the RAN. The Open RAN approach promotes the principles of openness, intelligence, virtualization, and interoperability. Leveraging intelligent network optimization and automation, Open RAN aims to elevate network performance and efficiency in the wireless network scenario [5].

Open RAN architectures, such as O-RAN, are distinguished by the separation of Hardware (HW) from Software (SW), allowing the utilization of Commercial Off-The-Shelf (COTS) HW from any vendor with the installation of any 5G functions SW, such as the DU. However, COTS HW tends to be less efficient compared to purpose-built HW, which is specifically tailored for RAN applications. To overcome this challenge, there is a need to optimize COTS HW by offloading the complex, real-time, and compute-intensive functions onto a dedicated HW accelerator. This optimization is crucial to enhance overall performance within the 5G O-RAN framework [6].

Reconfigurable computing facilitates the implementation of high-throughput HW kernels while maintaining software-like programmability, thanks to field-programmable technology. The modern SoPC integrates Field Programmable Gate Array (FPGA) fabric, with a hard multi-core Central Processing Unit (CPU) within a single chip. Such SoPCs empower the development of fully reconfigurable, high-performance (HP) systems. Hence, in this study we developed a Multi-Processor System-on-Chip (MPSoC) FPGA-enabled SmartNIC to host DU-RU functions and accelerate the next generation eNB (gNB) RU Low-PHY functions, paving the way for an acceleration platform for 5G O-RAN DU-RU functions. Specifically, the offloaded function is the Fast Fourier Transform (FFT) of Orthogonal Frequency Division Multiplexing (OFDM) symbols in uplink (UL) transmission.

The proposed implementation aims to explore the potential of Programmable Logic (PL) in accelerating FFT computations, optimizing the performance of gNB Low-PHY functions and deploying O-RAN 7-2x split DU and RU functions in one chip, hence avoiding data transfer from accelerators to CPU and viceversa. This solution serves as a trial to evaluate the

feasibility and benefits of using MPSoC FPGA acceleration in the context of network function offloading.

The study also emphasizes the establishment of a communication interface with the Processing System (PS), laying the groundwork for future enhancements and additional stack function implementations within the 5G communication protocol. We focus on the optimization of data communication to and from the FFT Intellectual Property (IP), with the assessment of communication overheads. By using a heterogeneous system that consists of a processor and reconfigurable HW, we build up a computing platform in which we can program the functionalities of the Low-PHY layer to adapt to evolving protocols.

II. RELATED WORKS

An overview of HW acceleration techniques has been provided in [7], where issues and solutions for improving 5G low-PHY functions computation time have been discussed. In [8] it is reported that an optimized FPGA-based implementation of FFT and Cyclic-prefix removal surpasses a CPU in processing time for large OFDM symbol sizes but also consumes less power. In [6] an implementation of a FPGA-accelerated SmartNIC was proposed, showing that for higher size point FFT processing the developed FPGA implementation outperformed the CPU-based one with up to 1.91x reduction in processing time and lower power consumption when compared to GPU-based Low-PHY function processing. However, CPU to FPGA communication affected computation time performance, hence contributing to the overall system latency increase, although a high data rate interface was used. In [9] data exchange between PS and PL have been evaluated, where it was proved that for large data transfers, external DDR memory is the best option. In [10] multiple analyses of parameters within the memory sub-system of MPSoCs have been performed, aiming to assist in the design of accelerators that offer HP. The authors extensively evaluate the memory performance and behavior for various in-chip communication port combinations, burst sizes, access patterns, and the number of accelerators per interface port. Following their findings related to the difference in experimental and theoretical performance, we have extensively tested our approach to establish experimental profiling foundations.

III. PROPOSED APPROACH

This study proposes the implementation of a DU-RU in an MPSoC FPGA. This implementation is highly programmable and different RU and DU functions can be flexibly allocated. The proposed SmartNIC can be plugged into any COTS-based white box as a client interface.

A. Selected Functional Split

3GPP defines split options to separate 5G stack modules in the RAN and connect them with open interfaces. Functional split disaggregates RAN into RU, DU, and Central Unit (CU). Consequently, different architectures for 5G RAN have emerged based on how to split these RAN functions, where

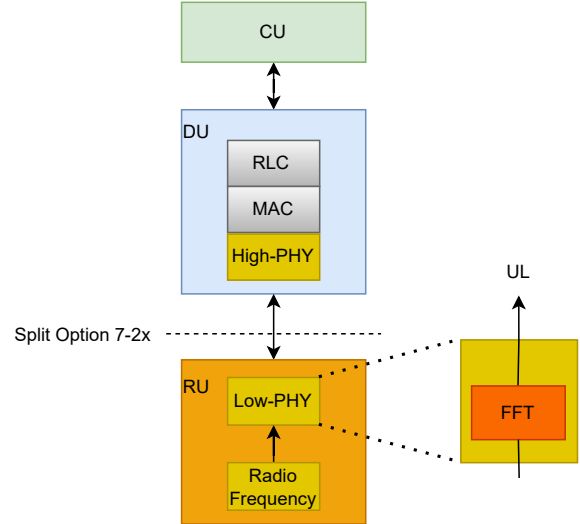


Fig. 1: O-RAN 7-2x functional split architecture.

to place them, and which interfaces to use to interconnect them. Hence, various options can be utilized to target different use cases and application demands [11]. There are 8 different options, each splitting the processing of the protocol stack functionalities on different HW and SW platforms. Open RAN (O-RAN) is based on option 7, more specifically, option 7-2x.

This split option divides the physical layer into high and low ones. In the uplink, the RU undertakes all of the low-PHY functions which include FFT. The remaining UL PHY processing, encompassing tasks like channel estimation, equalization, and decoding, is delegated to the DU, along with MAC and RLC processing. Fig. 1 shows the details.

In addition to the flexibility, RAN disaggregation allows network operators to tailor their networks to their specific needs. It also enables higher performance, which is benefited by the freedom to place functions on the computing platform that best suits them. In particular, most low-PHY functions are computationally demanding, need real-time processing, and are qualified to be implemented on specific purpose HW to improve performance.

B. MPSoC FPGA Enabled SmartNIC

The considered implementation of the DU-RU SmartNIC with split option 7-2x in the uplink direction is shown in Fig. 2. In the proposed system we develop a MPSoC FPGA-based SmartNIC equipped with an in-chip hard CPU. The MPSoC FPGA in this setup receives input signals from the Radio Frequency (RF) unit to perform low-PHY processing on the FPGA, then data is sent to the MPSoC CPU to perform DU functions and, then, sent through the board interface to the CU.

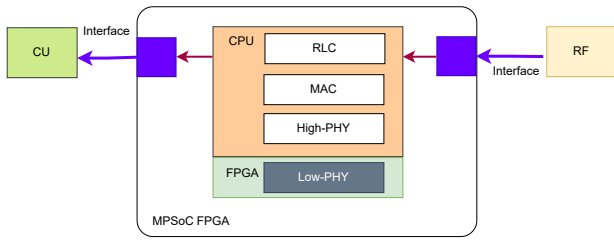


Fig. 2: Proposed MPSoC FPGA-based DU-RU SmartNIC.

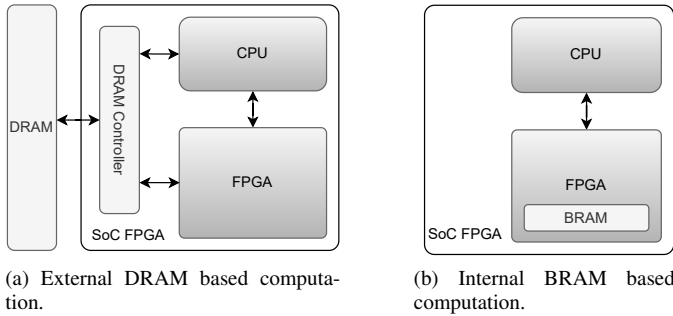


Fig. 3: Proposed architectures.

C. Low-PHY Acceleration

According to [12], a significant challenge faced by High-Performance (HP) systems is the memory wall, where accelerators frequently suffer from insufficient memory throughput and high latency. [13] also highlights that the effectiveness of SW/HW solutions depends on various interconnected factors like the amount of processed data and the communication port types. Hence, we meticulously evaluate the optimal memory subsystem for implementing an accelerated Low-PHY, introducing two distinct architectures tailored to the storage of input and output FFT samples. In the first solution, external off-chip Dynamic Random Access Memory (DRAM) is employed for data storage, establishing a connection to the FPGA based FFT module via a DRAM Controller as Fig. 3a shows.

The second solution, as depicted in Fig. 3b, employs internal on-chip Block RAM (BRAM) within the FPGA chip. This internal memory is utilized for both storing input samples and capturing output results generated during the FFT process. In both designs, the CPU does not intervene in memory access operations, as the FPGA has direct access to the memory resources. The CPU is leveraged for lightweight tasks, like performance monitoring and debugging purposes, but without effect on the main computation process.

D. Implementation Details

We target the Zynq Ultrascale+ MPSoC FPGA family. This heterogenous architecture features a dual-core ARM Cortex A-53 on the PS side accompanied by an HP, multi-ported DDR Controller. This controller facilitates shared access to a common external DDR memory between the PS and the PL, operating at speeds of up to 2667 Mbps. bus to facilitate

high-bandwidth data transfer between PS and the PL. In this setup, SW tasks are executed on the processor, while the PL accommodates custom HW modules. HW Accelerators within the PL are activated by SW tasks through AXI requests whenever HW accelerated processing is required. We leveraged the AMD LogiCORE IP FFT core, which implements the Cooley-Tukey FFT algorithm. For the forward FFT, the output sequence $X'(k)$, computed by the core is defined as

$$X'(k) = \frac{1}{s} \sum_{n=0}^{N-1} x(n) e^{-jnk \frac{2\pi}{N}},$$

where $k = 0, \dots, N-1$.

The input and output data samples are in IEEE-754 single precision (32-bit) Floating Point format. Output order is set to Natural Order, with the target clock frequency of the block set to 250 MHz. Fig. 4 shows the developed design. The Data Input channel `s_axis_data` of the FFT block utilizes an AXI-4 streaming interface with 64-bit data fields. Within each 64-bit data word, the less significant 32 bits carry the real part of the sample, while the most significant 32 bits contain the imaginary part. Simultaneously, the Data Output channel `s_axis_data` operates as a master AXI-4 streaming interface with identical characteristics to the input channel. Both interfaces are connected to the AXI Direct Memory Access (DMA) module through the Memory-Mapped to Streaming MM2S and Streaming to Memory-Mapped S2MM interfaces, respectively. This connection allows access to memory-mapped regions, facilitating the reading of input samples and writing of output samples. Both interfaces are configured to support a 64-bit data width with a burst size of 256 Bytes.

Our proposed implementation of FFT caters to various sizes, with input and output samples being read from and written to a memory block. We explore two distinct architectures. In the initial architecture, input samples reside in external DDR memory, transferred to the PL via the HP coherent HPC slave port, which provides a high bandwidth data path from the master FFT module to the DDR operating at a clock frequency of 1067 MHz. The DMA is connected to the HPC port through an AXI Smart Interconnect block, where the channel `M_AXI_MM2S` reads input samples from the specified memory address of the DDR memory, and the channel `M_AXI_S2MM` sends FFT computed and streamed output samples to DDR starting from the base address for writing. The `AXI_LITE` port of the DMA is a memory-mapped interface, it is connected to the HP master port `HPM` through an AXI Interconnect module to receive control signals from the PS. Simultaneously, the PL operates automatically at a frequency of 100 MHz, and for subsequent setups, we configure it to 187 and 300 MHz.

In the alternative approach, samples are placed in a BRAM of the PL. We use the AMD Block Memory Generator module, which is an advanced memory constructor that generates area and performance-optimized memories using embedded block RAM resources in FPGAs. It supports AXI4 and AXI4-Lite interface protocols and has a performance of up to 300 MHz

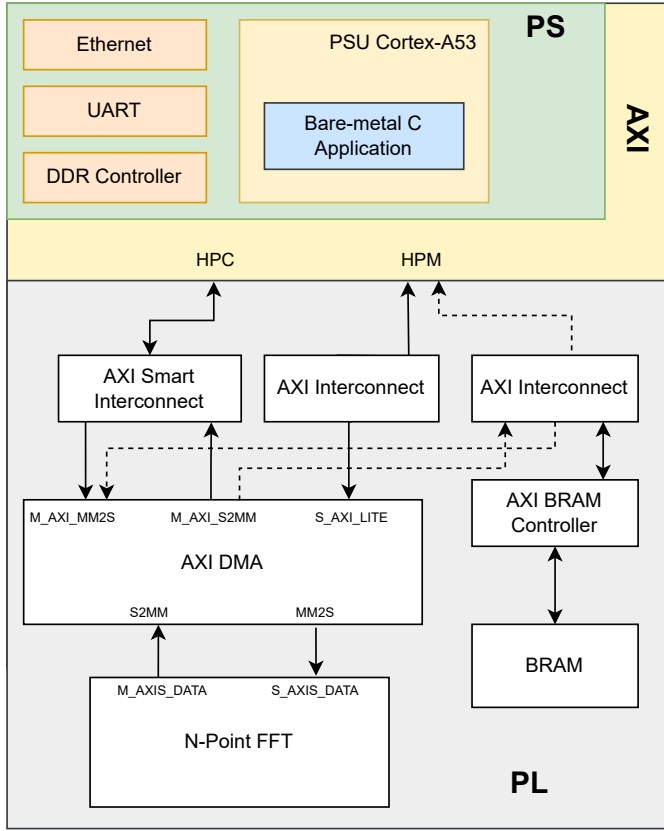


Fig. 4: Developed N-Point FFT on UltraScale+ MPSoC architecture.

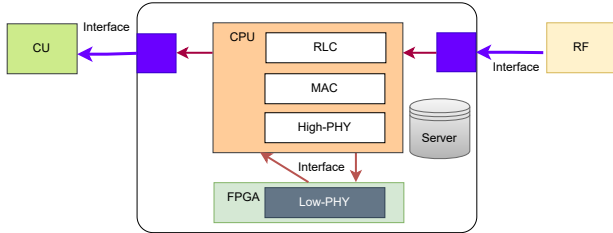


Fig. 5: COTS server equipped with FPGA.

and a maximum burst size of 256 bytes. It is configured as a single port RAM, has a 64-bit port for read and write and it is of 1024 depth. It is connected to the DMA through the Advanced eXtensible Interface (AXI) BRAM Controller. The AXI BRAM Controller is connected to the PS HP port HPM through an AXI Interconnect module, this enables control of the BRAM from the PS.

IV. EXPERIMENTAL RESULTS.

We leverage the capabilities of the Zynq UltraScale+ ZCU102 board, which embodies the features of the Zynq UltraScale+ MPSoC. We chose the Cortex-A53 64-bit quad-core processor which operates at 1.2 GHz frequency for developing our bare-metal C application in Vitis SW platform. The bare-metal SW stack gave us a simple, single-threaded

environment where C code runs directly on the ARM processor without the presence of an operating system, hence developing the necessary functionality with the least overhead. The bare-metal C Application developed enables to write input samples to the address of the memory device, in addition to launching FFT computation and specifying the memory region in which output results will be stored. Furthermore, we utilize it for monitoring the performance of the different developed implementations. The board is connected to a laptop through the UART interface for real-time performance monitoring and debugging. Source scripts of the HW design files, C applications, and Python codes are available on the project GitHub repository [14].

A. Benchmark Implementation: COTS Server with FPGA Off-loading

In [6], a COTS server with FPGA implements a DU unit, where the FPGA hosts the Low-PHY function, as shown in Fig. 5. The server's CPU sends the time domain samples received from RU to the FPGA via an interface. After processing in FPGA, IQ samples in the frequency domain return to the CPU via the same interface. Subsequently, the CPU handles the RLC, MAC, and High-PHY functions before transmitting to the CU. We implement also this scenario to benchmark the performance of our developed MPSoC FPGA-based SmartNIC solution.

To implement the benchmarking system we establish a connection between the board and a laptop acting as the server's CPU. The interface utilized is the PS Ethernet block GEM3 with the PS PHY, operating at 1000 Mbps. We developed a TCP server application on the board, while the laptop served as the client. A Python script, utilizing the socket library, is developed to format the generated FFT samples into TCP packets send them, and receive results back from the board. We conduct timing measurements for this setup and compare it with our proposed solution.

B. Resource Consumption

Resource utilization is extracted from the implementation report generated by AMD Vivado IDE, which was the tool used to synthesize the design and generate the bitstream. The values in Table I show the distribution of various resources. Notably, the BRAM-based approach exhibits higher utilization in some aspects compared to the DRAM-based implementation. BRAM's higher resource usage stems from its integration within the FPGA fabric, consuming more LUTs and FFs. Additionally, it needs an AXI controller for managing data access, also utilizing PL resources. In contrast, the DRAM-based design relies on built-in DDR and DDR controllers, reducing the demand for FPGA resources.

C. Power Consumption

Power consumption details of the developed designs were extracted from Vivado IDE power analysis reports. The values provided in Table II include both static and dynamic power consumption for the two designs. Static power refers to the

TABLE I: Resource utilization for BRAM and DRAM-based 1024-point FFT

Resource	DRAM-based	BRAM-based
Look-Up Table	13768	18652
LUTRAM	2050	2629
Flip-Flops	20731	26370
BRAM	30	55.50
Digital Signal Processors	18	18

power consumed when the device is configured but inactive, while dynamic power represents the power required during application execution and switching activities. The DRAM-based design shows extra power consumption both for PS and PL regions. In both designs the PS proves to be power hungry compared to the PL with around 90% of the total consumption, this brings consideration to the need for implementing the most possible tasks on the PL side for maximum power efficiency. The extra power consumption experienced when using the DRAM is explained by the involvement of both PS and PL in accessing the DDR memory through the HP port with high bandwidth.

TABLE II: Power consumption estimation

power		DRAM-based design	BRAM-based Design
Static	PS	0.098 W	0.098 W
	PL	0.625 W	0.625 W
Dynamic	PS	2.743 W	2.739 W
	PL	0.321 W	0.239 W

D. Performance Analysis of Low-PHY Processing

The execution time of different modules was measured with the C function `XTime_GetTime`. The measurement started when the DMA initiated the first data read and concluded after the last data write to the memory. The first experiment was carried out with a PL clock frequency of 99.99 MHz, the results shown in Table III indicate that for small FFT sizes, the BRAM-based implementation demonstrates slightly faster computation time, as BRAM excels when dealing with smaller data volumes due to its simpler access mechanisms. However, as the FFT size increases, the DRAM provides better performance with around 15.45% less time. As the size of FFT increases, the advantages of utilizing external DRAM become more apparent in terms of execution time. This is primarily due to the high frequency of DRAM, which facilitates higher throughput. The HP port associated with DRAM enables higher data transfer per clock cycle, making it optimal for processing large amounts of data efficiently.

TABLE III: N-point FFT computation execution time for different memory resources

FFT Size	DRAM Time (μ s)	BRAM Time (μ s)
8 Points	5.71	4.84
64 Points	9.37	8.37
256 Points	19.08	18.90
512 Points	31.48	32.00
1024 Points	52.22	60.29

The second set of experiments was carried out with a PL clock frequency of 187.42 MHz. The results presented in Table IV show that the DRAM performs better for most point sizes, and it keeps getting a lower timing over the BRAM implementation for larger point sizes, with a speedup of 11.82%.

TABLE IV: N-point FFT computation execution time for different memory resources with 187 MHz PL CLK FREQ

PL Clock Frequency	DRAM Time (μ s)	BRAM Time (μ s)
256 Points	10.66	11.09
512 Points	18.00	17.67
1024 Points	28.81	32.69

The experiments conducted at a clock frequency of 299.99 MHz revealed that the DRAM-based FFT implementation consistently outperformed the BRAM-based approach across all point sizes tested. This outcome underscores the effectiveness of DRAM, particularly at higher PL frequencies. As the PL frequency increases, DRAM can capitalize on its bandwidth capabilities more efficiently. Despite the overhead associated with accessing smaller data volumes, DRAM's superior bandwidth utilization allows it to deliver better performance across various FFT point sizes. In contrast, BRAM's limited bandwidth and capacity pose challenges as the PL frequency rises. The advantages of BRAM, become overshadowed by the superior bandwidth offered by DRAM and its HP port at higher frequencies.

TABLE V: Execution time for 300MHz PL clock FFT computation

Point Size	DRAM-based design	BRAM-based Design
256 Points	7.21	7.45
512 Points	11.14	11.42
1024	18.37	20.79

The comparison between DRAM and BRAM implementations within the context of FFT operations underscores the superiority of the former. The DRAM's performance in FFT computations is attributed to its operating frequency of 1067 MHz, significantly outpacing the maximum frequency of around 300 MHz for BRAM. Moreover, the presence of an HP port in the DRAM facilitates connectivity to the PL, thereby outperforming even the fact that the BRAM is PL-based. Furthermore, the limitation of BRAM in supporting the storage of large volumes of data underscores the indispensability of DRAM, which offers ample storage capacity. Moreover, the DRAM-based design is area-efficient with less resource utilization. The DRAM emerges as the optimal solution for Low-PHY functions acceleration, boasting performance, superior storage capacity, and efficient resource utilization for different design choices, thereby enabling high-speed computations.

E. Performance Analysis of MPSoC FPGA SmartNIC

As detailed in IV-A, our developed solution is compared against an FPGA-only offloading with external server CPU data communication. For both setups, we opted for the

DRAM-based FFT solution due to its demonstrated superior timing performance. Fig. 6 illustrates the results of these experiments. When employing an MPSoC FPGA, we observe approximately a 15x speedup in timing compared to utilizing an external CPU for input sample reception and transmission.

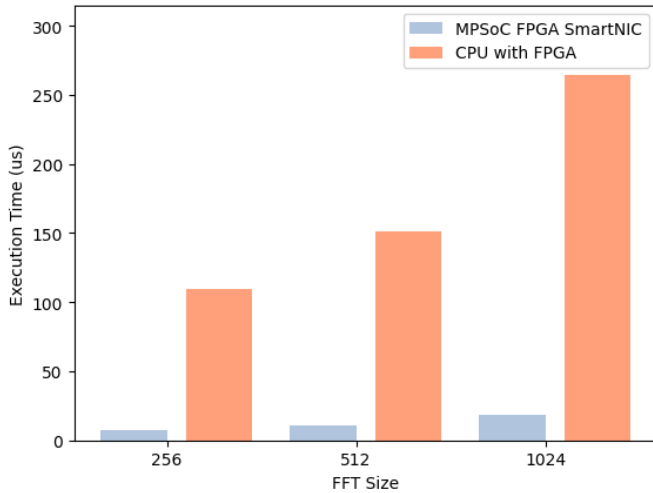


Fig. 6: Timing measurements for MPSoC FPGA-based SmartNIC vs CPU with FPGA

Our solution has significantly improved timing performance, primarily attributed to avoiding data exchange between an accelerator and processor through a peripheral interface. This was achieved through employing a heterogeneous system that combines processor and accelerator into a single chip, leading to a notable enhancement in computation compared to offloading the intensive computing tasks of the PHY layer to the FPGA while necessitating data transformation for performing additional processing externally. Employing a single-chip solution for sample exchanges reduces system latency and decreases the overall elaboration time needed to perform computation on an accelerator.

The developed implementation serves as a stepping stone for further acceleration of functions within the 5G protocol stack. The SW programmability of the PS enables the implementation of DU functions, by simply porting available codes like the OpenAirInterface C source codes. The same codes can be used to develop accelerated HW designs for RU functions using C/C++ High-Level Synthesis tools, leveraging parallelism directives to develop latency-critical functionalities. The prototype we have developed with the DMA being used to integrate an accelerated Low-PHY function IP into a SW-managed application can be exploited by other works to connect developed HW accelerated RU functions to SW-running DU functions or any other configurations based on the chosen split. Therefore, enabling further offloading of selected functions and the creation of an accelerated 5G stack that benefits from dynamic and runtime reconfiguration of HW functionality with SW ease of use. Additionally, this SmartNIC solution is pluggable, meaning it can be integrated into any

disaggregated device, (e.g., an Optical Network Termination — ONT — of a Passive Optical Network — PON) as a client interface. This level of integration offers advantages that a multi-chip-based solution cannot match, making it a superior option for accelerating 5G DU-RU functions.

V. CONCLUSION

In this work, we developed an MPSoC FPGA-based SmartNIC tailored to deploy DU-RU functions, with offloading Low-PHY processing to FPGA. We investigated the most suitable setup for minimizing Low-PHY function FFT computation time, where we found that utilizing an off-chip DRAM offers lower timing and resource utilization but with slightly higher power consumption compared to an internal BRAM-based solution. We proposed and implemented a model to deploy the O-RAN functional split which benefits from lower data transfer time thanks to the MPSoC FPGA device with an on-chip CPU and bare-metal application, hence achieving around 15x speedup in latency compared to deploying it on an FPGA-only device with server's CPU communication overhead.

REFERENCES

- [1] I. Parvez, A. Rahmati, I. Guvenc, A. I. Sarwat, and H. Dai, "A survey on low latency towards 5G: RAN, core network and caching solutions," *IEEE Commun. Surveys & Tut.*, vol. 20, no. 4, pp. 3098–3130, 2018.
- [2] S. Dang, O. Amin, B. Shihada, and M.-S. Alouini, "What should 6G be?" *Nature Electron.*, vol. 3, no. 1, pp. 20–29, 2020.
- [3] M. Polese, L. Bonati, S. D'oro, S. Basagni, and T. Melodia, "Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges," *IEEE Commun. Surveys & Tut.*, 2023.
- [4] L. Bonati, S. D'Oro, M. Polese, S. Basagni, and T. Melodia, "Intelligence and learning in O-RAN for data-driven NextG cellular networks," *IEEE Commun. Mag.*, vol. 59, no. 10, pp. 21–27, 2021.
- [5] X. Krasniqi, E. Hajrizi, and B. Qehaja, "Challenges and Lessons Learned during Private 5G Open RAN Deployments," in *Proc. 3rd Internat. Conf. on Electrical, Computer, Communications and Mechatronics Eng. (ICECCME)*. IEEE, 2023.
- [6] J. C. Borromeo, K. Kondepudi, N. Andrioli, and L. Valcarenghi, "FPGA-accelerated SmartNIC for supporting 5G virtualized Radio Access Network," *Computer Networks*, vol. 210, p. 108931, 2022.
- [7] —, "An overview of hardware acceleration techniques for 5G functions," in *Proc. 22nd Internat. Conf. on Transp. Opt. Netw. (ICTON)*. IEEE, 2020.
- [8] —, "Experimental evaluation of 5G vRAN function implementation in an accelerated edge cloud," in *Proc. 47th Europ. Conf. on Opt. Commun. (ECOC)*. IEEE, 2021.
- [9] J. Silva, V. Sklyarov, and I. Skliarova, "Comparison of on-chip communications in Zynq-7000 all programmable systems-on-chip," *IEEE Embedded Syst. Lett.*, vol. 7, no. 1, pp. 31–34, 2015.
- [10] K. Manev, A. Vaishnav, and D. Koch, "Unexpected diversity: Quantitative memory analysis for Zynq Ultrascale+ systems," in *Proc. Internat. Conf. on Field-Programm. Tech. (ICFPT)*. IEEE, 2019, pp. 179–187.
- [11] L. M. P. Larsen, A. Checko, and H. L. Christiansen, "A Survey of the Functional Splits Proposed for 5G Mobile Crosshaul Networks," *IEEE Commun. Surveys & Tut.*, vol. 21, no. 1, pp. 146–172, 2019.
- [12] M. Göbel, A. Elhossini, C. C. Chi, M. Alvarez-Mesa, and B. Juurlink, "A quantitative analysis of the memory architecture of FPGA-SOCS," in *Proc. 13th Internat. Symp. Applied Reconfig. Comput. (ARC 2017)*. Springer, 2017, pp. 241–252.
- [13] V. Sklyarov, I. Skliarova, J. Silva, and A. Sudnitson, "Analysis and comparison of attainable hardware acceleration in all programmable systems-on-chip," in *2015 Euromicro Conference on Digital System Design*. IEEE, 2015, pp. 345–352.
- [14] A. Bourenane. (2024) Source code of the implementation. [Online]. Available: <https://github.com/abdelghaniBourenane/5G-DU-RU-SmartNIC>